LAPORAN PRAKTIKUM
TEKNOLOGI IOT II

4.7 Tugas: Sistem Kendali Intensitas Cahaya (PWM Slider) via WebSockets

Dosen Pengampu:

1. Dr. Ir. Septafiansyah Dwi Putra - NIP : 198809112009121001
2. Agus Ambarwari, S.Pd., M.Kom. - NIP : 199208082019031020



Disusun oleh :

Ananda Haris Rahmadi (25782032)

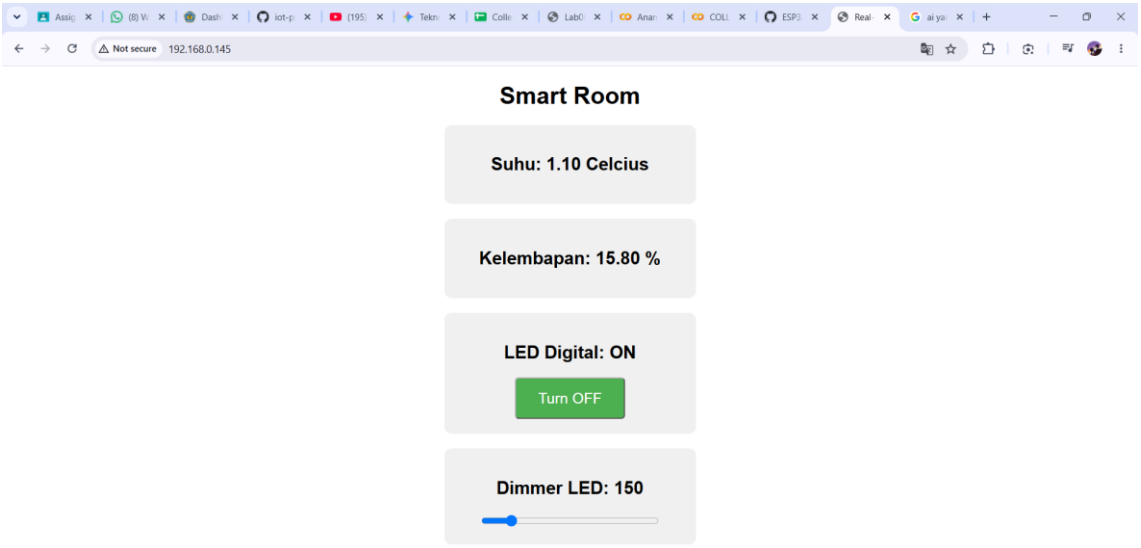
PROGRAM STUDI TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

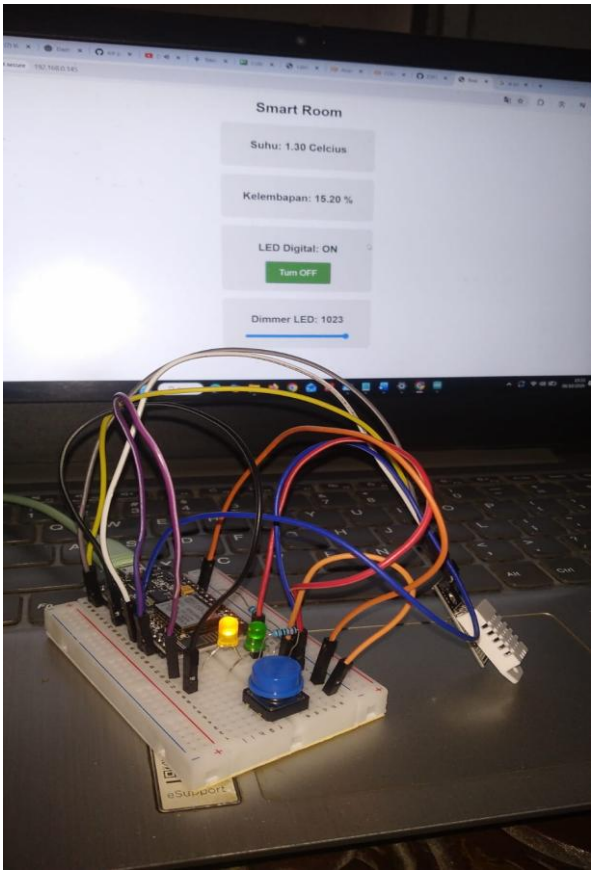
POLITEKNIK NEGERI LAMPUNG

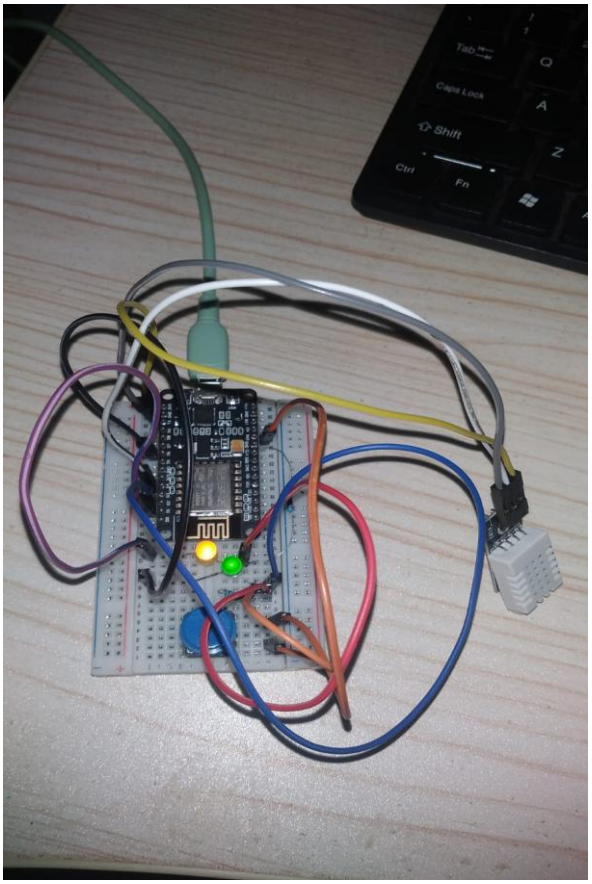
2026

Tampilan web



Alat dan web





Dalam Tugas 4.7 ini, saya membuat kontrol redup-terang LED (dimmer) berbasis **PWM** dan **WebSockets** secara *real-time*:

1. **Hardware:** Menambahkan LED baru di **Pin D1 (GPIO 5)** dengan resistor 220 Ohm tanpa melepas komponen lama.
2. **Front-End:** Menambahkan slider HTML (0-1023) yang mengirim string "pwm,<nilai>" via JavaScript sendPWM() saat slider digeser.
3. **Back-End:** NodeMCU memfilter pesan "pwm,", mengambil angka nilainya dengan substring(4), lalu mengeksekusinya pakai analogWrite().
4. **Hasil:** Kecerahan LED berubah instan tanpa *lag*, dan sistem tetap berjalan lancar berdampingan dengan tombol LED serta sensor **DHT22**.

Codenya:

```
#include <ESP8266WiFi.h>

#include <ESPAsyncTCP.h>
#include <ESPAsyncWebServer.h>
#include <DHT.h>

const char* ssid = "Network_eng";
const char* password = "d585de2865";

// Konfigurasi Pin
const byte dhtPin = 2;        // D4 (GPIO 2)
const byte buttonPin = 4;     // D2 (GPIO 4)
const byte ledPin = 12;       // D6 (GPIO 12) - LED Digital ON/OFF
const byte pwmLedPin = 5;     // D1 (GPIO 5)  - LED PWM Dimmer (Tugas
4.7)

DHT dht(dhtPin, DHT22);

// Variabel Pelacak Status (State & Cache)
bool ledState = false;
int pwmVal = 0;               // Variabel penampung nilai PWM
String currentTemp = "--";
String currentHum = "--";
int buttonState;
int lastButtonState = LOW;
unsigned long lastDebounceTime = 0;
const unsigned long debounceDelay = 50;
unsigned long lastTime = 0;

// Inisialisasi Async Web Server (port 80) & WebSocket (rute /ws)
AsyncWebServer server(80);
AsyncWebSocket ws("/ws");

// ---------------- HTML & JAVASCRIPT (FRONT-END) ----------------
const char index_html[] PROGMEM = R"rawliteral(
```

```
<!DOCTYPE html>
<html>
<head>
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Real-Time IoT Web</title>
  <style>
    body { font-family: Arial; text-align: center; }
    .card { background: #f0f0f0; margin: 20px auto; padding: 20px; max-
width: 300px; border-radius: 10px; }
    button { padding: 15px 30px; font-size: 20px; border-radius: 5px;
cursor: pointer; color: white;}
    .btn-on { background-color: #4CAF50; }
    .btn-off { background-color: #f44336; }
    .slider { width: 80%; cursor: pointer; }
  </style>
</head>
<body>
  <h1>Smart Room</h1>

  <div class="card">
    <h2>Suhu: <span id="tempValue">--</span> Celcius</h2>
  </div>

  <div class="card">
    <h2>Kelembapan: <span id="humValue">--</span> %</h2>
  </div>

  <div class="card">
    <h2>LED Digital: <span id="ledStatus">OFF</span></h2>
    <button id="toggleBtn" class="btn-off" onclick="toggleLed()">Turn
ON</button>
  </div>

  <!-- Kartu Dimmer Slider PWM (Tugas 4.7) -->
  <div class="card">
    <h2>Dimmer LED: <span id="pwmText">0</span></h2>
    <input type="range" min="0" max="1023" value="0" class="slider"
id="pwmSlider" oninput="sendPWM(this.value)">
  </div>

  <script>
    var gateway = `ws://${window.location.hostname}/ws`;
    var websocket;

    window.addEventListener('load', onLoad);
    function onLoad(event) { initWebSocket(); }
```

```
function initWebSocket() {
  websocket = new WebSocket(gateway);
  websocket.onopen    = onOpen;
  websocket.onclose   = onClose;
  websocket.onmessage = onMessage;
}

function onOpen(event) { console.log('WebSocket Terkoneksi'); }
function onClose(event) { setTimeout(initWebSocket, 2000); }

function toggleLed(){
  websocket.send('toggle');
}

// Fungsi pengirim nilai slider PWM real-time
function sendPWM(val){
  document.getElementById('pwmText').innerHTML = val;
  websocket.send('pwm,' + val);
}

function onMessage(event) {
  var dataObj = JSON.parse(event.data);

  if(dataObj.suhu !== undefined) {
     document.getElementById('tempValue').innerHTML = dataObj.suhu;
  }

  if(dataObj.hum !== undefined) {
     document.getElementById('humValue').innerHTML = dataObj.hum;
  }

  if(dataObj.led !== undefined) {
     var btn = document.getElementById('toggleBtn');
     var status = document.getElementById('ledStatus');
     if(dataObj.led == "1"){
       status.innerHTML = "ON";
       btn.innerHTML = "Turn OFF";
       btn.className = "btn-on";
     } else {
       status.innerHTML = "OFF";
       btn.innerHTML = "Turn ON";
       btn.className = "btn-off";
     }
  }

  if(dataObj.pwm !== undefined) {
     document.getElementById('pwmSlider').value = dataObj.pwm;
```

```
          document.getElementById('pwmText').innerHTML = dataObj.pwm;
        }
      }
    </script>
</body>
</html>
)rawliteral";

// ---------------- BACK-END & WEBSOCKET LOGIC ----------------

void notifyClients() {
  String jsonString = "{\"led\":\"" + String(ledState ? 1 : 0) + "\", ";
  jsonString += "\"suhu\":\"" + currentTemp + "\", ";
  jsonString += "\"hum\":\"" + currentHum + "\", ";
  jsonString += "\"pwm\":\"" + String(pwmVal) + "\"}";
  ws.textAll(jsonString);
}

// Handler pesan masuk dari browser
void handleWebSocketMessage(void *arg, uint8_t *data, size_t len) {
  AwsFrameInfo *info = (AwsFrameInfo*)arg;
  if (info->final && info->index == 0 && info->len == len && info->opcode
== WS_TEXT) {
    data[len] = 0;
    String message = String((char*)data);

    // 1. Eksekusi Perintah Toggle Button
    if (message == "toggle") {
      ledState = !ledState;
      notifyClients();
    }

    // 2. Eksekusi Parsing String Perintah PWM Slider ("pwm,VAL")
    else if (message.startsWith("pwm,")) {
      String valStr = message.substring(4); // Mengambil karakter setelah
"pwm,"
      pwmVal = valStr.toInt();              // Konversi teks angka ke
integer
      analogWrite(pwmLedPin, pwmVal);       // Eksekusi tingkat kecerahan
LED
      notifyClients();
    }
  }
}

void onEvent(AsyncWebSocket *server, AsyncWebSocketClient *client,
AwsEventType type,
```

```
             void *arg, uint8_t *data, size_t len) {
  switch (type) {
    case WS_EVT_CONNECT:
      Serial.printf("Client WebSocket #%u terhubung\n", client->id());
      notifyClients();
      break;
    case WS_EVT_DISCONNECT:
      Serial.printf("Client WebSocket #%u terputus\n", client->id());
      break;
    case WS_EVT_DATA:
      handleWebSocketMessage(arg, data, len);
      break;
  }
}

void setup() {
  Serial.begin(115200);

  pinMode(buttonPin, INPUT);
  pinMode(ledPin, OUTPUT);
  digitalWrite(ledPin, LOW);

  // Inisialisasi Pin PWM LED (D1 / GPIO 5)
  pinMode(pwmLedPin, OUTPUT);
  analogWrite(pwmLedPin, pwmVal);

  dht.begin();

  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print(".");
}
  Serial.println("\nIP Address: " + WiFi.localIP().toString());

  server.on("/", HTTP_GET, [](AsyncWebServerRequest *request){
    request->send_P(200, "text/html", index_html);
  });

  ws.onEvent(onEvent);
  server.addHandler(&ws);
  server.begin();
}

void loop() {
  ws.cleanupClients();

  digitalWrite(ledPin, ledState ? HIGH : LOW);
```

```
  // Baca Tombol Fisik
  int reading = digitalRead(buttonPin);
  if (reading != lastButtonState) {
    lastDebounceTime = millis();
  }
  if ((millis() - lastDebounceTime) > debounceDelay) {
    if (reading != buttonState) {
      buttonState = reading;
      if (buttonState == HIGH) {
        ledState = !ledState;
        notifyClients();
      }
    }
  }
  lastButtonState = reading;

  // Baca Sensor DHT11 tiap 3 detik
  if ((millis() - lastTime) > 3000) {
    float t = dht.readTemperature();
    float h = dht.readHumidity();

    if(!isnan(t) && !isnan(h)) {
      currentTemp = String(t);
      currentHum = String(h);
      notifyClients();
    }
    lastTime = millis();
  }
}
```